Messaging module: what was built and how to use it

announce-reactive, worktree `messaging` (branch `feat/messaging`, commit f138751d), 26 September 2026

This guide is for the developer who merges the branch into `announce-jo-branch` and then runs, integrates or extends the chat service. Part 1 says what was delivered and why it is shaped the way it is. Part 2 is the step-by-step procedure to apply it on a machine. Part 3 explains how clients and other services use it. The reference documents in the repository are `doc/messaging-context.md` (the full contract) and section 9 of `doc/order-context.md` (the deviations from the original sketch).

Part 1. What was done

1.1 The assignment

The worktree block in CLAUDE.md asked for a new bounded context, **messaging**, built like announce-management on the cqrs-reactive framework: a **Conversation** aggregate on EventSourcingDB, read-model tables in MySQL fed by projections, WebSocket delivery fanned out across nodes through Kafka, the public event **OfferAcceptedV1** on topic `messaging.v1` for the order context, a subscription to `order.v1` that posts system messages, and a support queue with staff bulletins.

1.2 What is in the branch

Area	Delivered
Schema	`db-reactive/sql/announce_messaging.sql`: the four chat tables (conversation, participant, message, bulletin) moved out of `announce_auction_chat.sql`, reshaped with varchar(36) ids, `announce_id`, a per-conversation message `seq`, offer columns and `last_event_id`. The build script and the generated bootstrap combine three source files.
Mappers	Four reactive MyBatis mappers in db-reactive with hand-written statements: `upsert`, `updateOffers`, `selectByParticipant`, `raiseLastReadSeq`, `updateOfferStatus`, `selectAfter`; a live-database mapper test.
messaging-api	The published language: `OfferAcceptedV1` and `ConversationOpenedV1`, annotated with `@PublicEvent`.
messaging: domain	`Conversation` state, commands, seven top-level events, `ConversationRules` and explicit OpenCQRS handler definitions.
messaging: application	Use cases (`Conversations`), queries over the read model, bulletins, the `AnnounceLookup` port and the live-record types.
messaging: infrastructure	Read-model projection (group `conversation-read-model`), relay to `messaging.v1`, live relay to `messaging.live`, per-node live subscriber and session registry, `order.v1` subscriber with local stubs, announce API client.
messaging: interfaces	REST under `/api/conversations` and `/api/admin`; WebSocket handlers at `/ws/conversations/{id}` and `/ws/staff`.
Tests	20 tests: aggregate rules through `CommandFixture`, a REST slice, and an end-to-end test with EventSourcingDB and Kafka from Testcontainers plus the live MySQL read model.
Docs	`doc/messaging-context.md`; section 9 of `doc/order-context.md`; the topic list in `doc/cqrs-reactive.md`.

1.3 Design decisions worth knowing

- **One conversation per announce and buyer.** The id is derived: `UUID.nameUUIDFromBytes("listing:" + announceId + ":" + buyerId)`. Opening the pair a second time returns the existing conversation without writing an event, and any other service (the order subscriber) can address the thread without a lookup.
- **The seller comes from announce-management.** The aggregate needs the seller to make them a participant; the `AnnounceLookup` port calls `GET /api/announces/{id}`. When the `announce-management-api` module is merged, this can become a snapshot fed from `announce.v1`.
- **Read markers are not events.** MarkRead writes `last_read_seq` on the participant row. Messages carry a `seq`, so unread = last message seq minus last read seq, and the history cursor is `after=<seq>`.
- **Offers.** Only the buyer of a listing conversation makes an OFFER; a new offer supersedes the open one; the seller accepts or declines the latest; after an acceptance no further offer is taken.
- **Live fan-out reuses the framework relay.** A second relay group, `messaging-live`, maps every visible event to a record on topic `messaging.live`, keyed by the conversation. Each node subscribes with its own consumer group and pushes to the sockets it holds. Delivery is at least once; clients de-duplicate on the message id.
- **Bulletins are plain rows** plus a live record; they are not event-sourced.
- **Ids.** Conversation and message ids are UUID strings; member and staff ids stay integers (`announce_user.id`, `announce_admin.id`), so the public event carries integer buyer and seller ids.
- **Headers instead of security.** `X-User-Id` identifies a member, `X-Staff-Id` a staff member, `X-Command-Id` makes a write idempotent. The gateway is trusted; the service is not exposed directly.

Part 2. How to apply it, step by step

2.1 Merge the branch

Run from the main checkout, the one that has `announce-jo-branch` checked out (a worktree cannot merge into a branch checked out elsewhere):

```
cd /home/bimeni/shopping_apps/announce_app/announce-reactive
git merge feat/messaging
```

The merge touches the parent pom (two new modules and one managed dependency), db-reactive, the docs and adds the two module directories. If the order-reactive branch is merged first, expect a textual conflict only in the parent pom's module list and in `db-reactive/sql/build_announce_reactive_db.sh`; keep both sides.

2.2 Bring the database up to date

The four chat tables changed shape (int ids became varchar(36), new columns). On the developer machine this was already done: the live `announce_reactive_db` had the tables empty, they were dropped and recreated. On any other database, run:

```
mysql -uannounce -p announce_reactive_db -e "DROP TABLE IF EXISTS announce_conversation,
  announce_conversation_participant, announce_chat_message, announce_bulletin;"
mysql -uannounce -p announce_reactive_db < db-reactive/sql/announce_messaging.sql
```

A fresh database gets everything from the generated bootstrap, which now includes the messaging section:

```
mysql -uroot -p < db-reactive/sql/announce_reactive_db.sql
```

The two framework tables (`cqrs_command_log`, `cqrs_readside_progress`) are created by the service on its first start-up.

2.3 Build in the right order

The service depends on db-reactive (the new mappers) and messaging-api. Install them first, then build the service:

```
mvn -N install                                        # once, so children resolve the parent
mvn -pl db-reactive,messaging-api install -DskipTests
mvn -pl messaging -DskipTests compile
```

Several worktree sessions share `~/.m2`. Another session can overwrite the installed db-reactive jar with one that lacks the conversation mappers. If a messaging build fails on missing `AnnounceConversationMapper` classes, re-run the install above or build with `mvn -pl messaging -am ...` so Maven uses this checkout's modules.

2.4 Start the infrastructure and the service

```
docker compose -f docker/cqrs/docker-compose.yml up -d    # EventSourcingDB :3000, Kafka :9092
mvn -pl announce-management -P '!webapp' spring-boot:run  # :8085, answers the seller lookup
mvn -pl messaging spring-boot:run                         # :8086
```

Configuration lives in `messaging/src/main/resources/config/application.yml`. The values that differ per environment have environment variables:

Variable	Default	Meaning
ANNOUNCE_R2DBC_URL	r2dbc:mysql://localhost:3306/announce_reactive_db	read model, checkpoints, command log

Variable	Default	Meaning
ANNOUNCE_DB_USER / ANNOUNCE_DB_PASSWORD	announce / (developer password)	database account
ANNOUNCE_ESDB_URI / ANNOUNCE_ESDB_TOKEN	http://localhost:3000 / secret	EventSourcingDB
ANNOUNCE_KAFKA_BROKERS	localhost:9092	Kafka bootstrap servers
ANNOUNCE_MANAGEMENT_URL	http://localhost:8085	announce-management, pinned outside Eureka

Other settings: `messaging.max-text-length` (4000) and `messaging.max-page-size` (200). Eureka registration is on by default, as in announce-management; disable it with `eureka.client.enabled=false` when running without the registry.

2.5 Run the tests

```
mvn -pl messaging test          # 20 tests
mvn -pl db-reactive test        # 56 tests, includes the new mapper test
```

The end-to-end test needs Docker (EventSourcingDB and Kafka containers) and the live MySQL on localhost:3306. Before the Spring context starts it deletes the checkpoints of the three projection groups and any leftover rows of earlier runs (conversations whose announce id starts with `e2e-`), because the checkpoints refer to event ids of a container that may have been recreated. It removes every row it writes when it finishes.

2.6 Follow-ups at integration time

- Replace the four records under `messaging/.../infrastructure/messaging/order` by a dependency on `order-api` once the order-reactive branch is merged. Field names and event types already match that module's `OrderPaidV1`.
- Add a gateway route for `/api/conversations/**`, `/api/admin/conversations/**`, `/api/admin/bulletins` and `/ws/**` (use `lb:ws://messaging` for the WebSocket routes). The gateway must set the identity headers.
- When `announce-management-api` exists, feed an announce snapshot from `announce.v1` and implement `AnnounceLookup` over it instead of the HTTP call.

Part 3. How to use it

3.1 Identity headers

Header	Who	Used on
X-User-Id	a member (announce_user.id)	every member route and the WebSocket handshake
X-Staff-Id	a staff member (announce_admin.id)	every /api/admin route, staff sockets; staff may also read any conversation
X-Command-Id	any caller	optional on writes; the same id replays the first result instead of running again

3.2 A buyer talks to a seller

Step 1: open (or find) the conversation about an announce. The response is the aggregate state; the read model follows a moment later.

```
curl -X POST http://localhost:8086/api/conversations \
  -H 'X-User-Id: 7' -H 'Content-Type: application/json' \
  -d '{"type":"LISTING","announceId":"<announce id>"}'
# 201 {"id":"<conversation id>","type":"LISTING","status":"OPEN","buyerId":7,"sellerId":8,
#      "participants":[{"type":"USER","id":7,"role":"BUYER"},
#                      {"type":"USER","id":8,"role":"SELLER"}], ...}
```

Errors: 404 `announce-not-found`, 409 `buyer-is-seller` when the owner tries to open a conversation about their own announce.

Step 2: load the history, then keep a socket open for what follows. History is never replayed on the socket.

```
curl http://localhost:8086/api/conversations/<id>/messages?after=0&limit=50 -H 'X-User-Id: 7'
```

Step 3: connect the socket and send frames. Every frame the client sends is answered with an `ack` or an `error`; every message of the conversation, including the client's own, arrives as a `message` frame.

```
ws://localhost:8086/ws/conversations/<id>      (headers X-User-Id: 7)

-> {"type":"post","clientMsgId":"c1","contentType":"TEXT","content":"Is it still available?"}
<- {"type":"ack","action":"post","messageId":"...","seq":1,"clientMsgId":"c1","duplicate":false}
<- {"type":"message","conversationId":"...","messageId":"...","seq":1,"senderType":"USER",
    "senderId":7,"contentType":"TEXT","content":"Is it still available?","clientMsgId":"c1",
    "sentAt":"..."}

-> {"type":"post","clientMsgId":"c2","contentType":"IMAGE","attachmentUrl":"https://.../photo.jpg"}
-> {"type":"offer","clientMsgId":"c3","amountCents":90000,"currency":"SEK",
    "content":"Would you take 900?"}
-> {"type":"read","seq":3}          -> {"type":"close"}          -> {"type":"ping"}
```

The seller answers the offer from their own socket (or over REST):

```
-> {"type":"accept-offer"}                          (or "decline-offer")
<- {"type":"ack","action":"accept-offer"}
<- {"type":"status","change":"OFFER_ACCEPTED","status":"OPEN","messageId":"...",
    "amountCents":90000,"currency":"SEK","participantType":"USER","participantId":8,"at":"..."}
```

Rules the socket enforces: a repeated `clientMsgId` posts once and the ack says `duplicate: true`; a seller sending an offer gets `{"type":"error","status":403,"code":"not-buyer"}`; a caller who is not a participant is closed with code 4403 at the handshake, one without a header with 4401. If the socket drops, reconnect and call the messages route with `after=` the last seq you saw.

Every socket frame has a REST equivalent for clients without WebSocket support:

Route	Body	Answer
POST /api/conversations/{id}/messages	{clientMsgId, contentType, content, attachmentUrl, amountCents, currency}	201 {messageId, seq, clientMsgId, duplicate}
POST /api/conversations/{id}/offers/accept | decline	none	the conversation state
POST /api/conversations/{id}/read	{seq}	204
POST /api/conversations/{id}/close	none	the conversation state
GET /api/conversations?page=1&size=20	none	the caller's conversations, latest activity first, with unread counts
GET /api/conversations/{id}	none	the conversation with participants, open and accepted offer

3.3 Support: a member asks, staff picks it up

```
# the member
curl -X POST http://localhost:8086/api/conversations -H 'X-User-Id: 7' \
  -H 'Content-Type: application/json' \
  -d '{"type":"SUPPORT","subject":"Where is my parcel?"}'          # 201, status WAITING

# staff: the queue, then join (WAITING -> OPEN), then write, block or close
curl 'http://localhost:8086/api/admin/conversations?status=WAITING' -H 'X-Staff-Id: 3'
curl -X POST http://localhost:8086/api/admin/conversations/<id>/join  -H 'X-Staff-Id: 3'
curl -X POST http://localhost:8086/api/conversations/<id>/messages -H 'X-Staff-Id: 3' \
  -H 'Content-Type: application/json' -d '{"clientMsgId":"s1","content":"How can I help?"}'
curl -X POST http://localhost:8086/api/admin/conversations/<id>/block -H 'X-Staff-Id: 3' \
  -H 'Content-Type: application/json' -d '{"reason":"spam"}'
```

The member may write while the conversation is WAITING. A staff member who joined is a participant like any other and can use the conversation socket. Staff can read every conversation and close any of them; only staff can block.

3.4 Bulletins for the staff dashboard

```
ws://localhost:8086/ws/staff?clientType=desktop     (headers X-Staff-Id: 3)

curl -X POST http://localhost:8086/api/admin/bulletins -H 'X-Staff-Id: 3' \
  -H 'Content-Type: application/json' \
  -d '{"audience":"STAFF","clientType":"desktop","content":"Queue is long, please pick up",
       "expiresAt":"2026-09-27T18:00:00"}'
<- {"type":"bulletin","id":12,"adminId":3,"audience":"STAFF","clientType":"desktop","content":"...",
    "at":"..."}

curl 'http://localhost:8086/api/admin/bulletins?audience=STAFF' -H 'X-Staff-Id: 3'
```

A bulletin with a client type reaches the staff sockets of that type; one without reaches every staff socket on every node.

3.5 Integration with the order context

Direction	Topic	Record	What happens
messaging to order	messaging.v1	OfferAcceptedV1 (type messaging.offer-accepted, v1): conversationId, announceId, buyerId, sellerId, amountCents, currency, acceptedAt	order creates the deal at the offered amount (CreateFromOffer)
messaging to anyone	messaging.v1	ConversationOpenedV1 (messaging.conversation-opened)	informational
order to messaging	order.v1	OrderAgreedV1, OrderPaidV1, OrderShippedV1, OrderSettledV1 (types order.agreed, order.paid, order.shipped, order.settled)	a SYSTEM message in the listing conversation of buyer and seller; the conversation is opened when the deal had no chat

Consumers of `messaging.v1` depend on the `messaging-api` module and use the framework's `PublicEventSubscribers`:

```
@Bean
Consumer<Message<byte[]>> messagingEvents(PublicEventSubscribers subscribers, Orders orders) {
    return subscribers.forTopic("messaging")
            .on(OfferAcceptedV1.class, (event, headers) ->
                    orders.createFromOffer(event, headers.eventId()))
            .build();
}
# application.yml of the consumer
spring.cloud.function.definition: messagingEvents
spring.cloud.stream.bindings.messagingEvents-in-0.destination: messaging.v1
spring.cloud.stream.bindings.messagingEvents-in-0.group: order
```

Delivery is at least once on every topic. The messaging subscriber de-duplicates order events by using the Kafka event id as the client message id of the SYSTEM message; consumers of `messaging.v1` should de-duplicate on `headers.eventId()` the same way.

3.6 Operations

- **Rebuilding the read model.** `ProjectionRebuilder.rebuild("conversation-read-model")` empties the three tables and replays the log. Read markers are lost by a rebuild; every other column is recomputed.
- **Checkpoints.** One row per group (`conversation-read-model`, `messaging-public`, `messaging-live`) in `cqrs_readside_progress`. Never point a database with checkpoints at a different EventSourcingDB; clear the three rows first.
- **Consumer groups.** Each node creates a live consumer group named `messaging-live-<uuid>`; Kafka expires unused groups after its offset retention. This is expected.
- **Error format.** Every REST error is an RFC 7807 problem detail with a `code` property; socket errors carry the same code and status inside the `error` frame.